Attention Is All You Need

1. sequence transduction 序列转换，指机器翻译入序列转换成出序列任务统称
2. 本文提出了一种只基于**注意力机制**的新型网络架构
3. 什么是卷积神经网络CNN和循环神经网络RNN
4. BLEU这个指标是怎么计算的
5. 这个模型在英语转德语的翻译任务中达到了很好的效果

BLEU分数怎么算?

1. n-gram 意思是连续n个词的片段，n-gram 是连续滑动切出来的片段
    1. 1-gram是一个词
    2. 2-gram是连续两个词
    3. 3-gram是连续三个词
    4. 4-gram是BLEU中常常使**用到4**
    5. 举个例子：我是小明
        1. 1-gram： 我 是 小 明
        2. 2-gram： 我是 是小 小明
        3. 3-gram： 我是小 是小明
        4. 4-gram： 我是小明
2. 精确率 对于每个n-gram计算

$$P_n = \frac{\text{机器翻译中出现在参考译文里面的}n-gram\text{总数}}{\text{机器翻译中所有}n-gram\text{总数}}$$

3. 加权几何平均

$$e^{\frac{1}{4}\sum_{n=1}^{4} logP_n}$$

4. 惩罚过短翻译，如果机器翻译得特别短，分数会比较高，所以为了抑制这种情况，所以加一个惩罚

$$BP = \begin{cases} 1 & \text{机器长度} \geq \text{参考长度} \\ e^{1-\frac{\text{参考长度}}{\text{机器长度}}} & \text{否则} \end{cases}$$

5. 最终得BLEU公式

$$BLEU = BP \times e^{(\frac{1}{4}\sum_{n=1}^{4} logP_n)}$$

6. 范围0-100 越高越好（最后会自动乘一个100）

token的向量计算

这个比较有意思！

正常的自然语言计算机是不可能会读懂的，所以一定要转成数字，而数字终究是有限的，怎么表示很多很多的数呢，就用数字位的笛卡尔积的感觉，就是向量

而这里的token对应的向量就是一个词语在计算机里面的表示，而我们知道一个词语在不同的语境会有不同的意思，那么在初始化的时候 可能会根据不同的意思出现的次数赋予不同的权重，然后结合这个词语在这个语境下和其他token的计算来修正这个token的向量使它更加偏向某一个意思的方向，那么这样也就可以认为计算机理解了这个词语了

“北京” - “中国” + “法国” ≈ “巴黎”

CNN卷积神经网络Convolutional Neural Network

1. CNN主要用途是用来识别图片
2. 什么是卷积，什么是卷积核?

   卷积是指一个操作，其中用到卷积核。

   卷积核的本质是一个二维的矩阵，和输入的图片的相同的大小的像素块做**相**

乘并相加的操作，然后输出的一个数作为特征矩阵中对应位置的值，这个**操作**叫**卷积。**

$$output[i][j] = \sum_{k=0}^{m-1}\sum_{l=0}^{m-1} input[i+k][j+l] \times kernel[k][l]$$

注意：

- 矩阵是从(0,0)开始的
- 一个**n*n**的像素块对一个**m*m**的卷积核做卷积操作，得到的是一个**(n-m+1)*(n-m+1)**的特征像素块
- 如果是**m*n**的像素块对一个**x*y**的卷积核做卷积，那么就是**(m-x+1)*(n-y+1)**的特征像素块
- 因为彩色图片是由RGB三种颜色组成的，因此处理的时候一般会分成三个通道，用多个卷积核在图片上滑动，得到多张特征图，然后将这些特征图重叠相加，得到最终这一个卷积核对应的特征图。

3. 什么是激活函数?
    1. 输入：特征图中的每个位置的数值
    2. 输出：经过函数处理之后的对应值
    3. 目的
        - 引入非线性函数，增强表达
        - 对特征做筛选，增强表达
        - 梯度传播与训练稳定性
    4. 常见的激活函数
        - 隐藏层常用
            - ReLU f(x) = max(0,x)
        - 输出层常用
            - 二分类：$\sigma(x) = \frac{1}{1+e^{-x}}$
            - 多分类：$softmax(z_i) = \frac{e^{z_i}}{\sum\limits_j e^{z_j}}$

    注：输入层，隐藏层和输出层

- 输入层是接受原始数据，根据这个无法直接做出判断
- 隐藏层是出于二者中间，对数据进行处理直到可进行判断的过程
- 输出层就是把最后的特征变为预测结果

4. 什么是池化?
    1. 最大池化Maxpool
    2. 平均池化Avgpool

       上面这两种分别是对这个滑动块中取最大值或者平均值来实现池化的
    3. 全局平均池化Global Average Pool

       这个就是对一张图所有的数取平均编程一个数就叫GAP
5. 什么是全连接层 linear
    1. linear是全连接层，把向量做加权求和
        1. $y = Wx + b$

           W是1*256权重矩阵

           b是偏置

           x是多维（256*1）向量

           y是一个具体的数字
        2. $y = \sum_{i=1}^{256} W_i, x_i + b$这个y经过一个sigmoid函数就变成了预测结果$p = \sigma(y) = \frac{1}{1+e^{-y}}$
    2. **这里有个待解决的问题就是为什么y这么处理就能得到对应的预测的概率呢?**

$$logit = log\frac{p}{1-p}$$

$$s = 0\text{那么就是觉得二分的两种情况差不多}$$

$$s > 0\text{那么偏向于正类}$$

$$s < 0\text{那么偏向于负类}$$

$$logit\text{就是}s\text{也就是}y$$

通俗的理解感觉就是有这么几个特征，根据这些特征对图片进行打分，根据打分情况偏向于哪边，来预测对应的概率是多少

6. 过程示例

    1. Block1：Conv3×3, C=32, s=1, p=1 → BN → ReLU → Conv3×3, C=32, s=1, p=1 → BN → ReLU → MaxPool2×2
       输出形状：112×112×32
    2. Block2：Conv3×3, C=64 → BN → ReLU → Conv3×3, C=64 → BN → ReLU → MaxPool2×2
       输出形状：56×56×64
    3. Block3：Conv3×3, C=128 → BN → ReLU → Conv3×3, C=128 → BN → ReLU → MaxPool2×2
       输出形状：28×28×128
    4. Block4：Conv3×3, C=256 → BN → ReLU → Conv3×3, C=256 → BN → ReLU → GlobalAvgPool
       输出形状：1×1×256 → 展平成 256 维向量
    5. 分类头：Dropout(0.5) → Linear(256→1) → Sigmoid（输出“是猫”的概率）

解释：Block是多个层的组合单元，一个箭头两边分别是一个层

C是输出通道数，也就是说这层有多少卷积核进行了卷积操作

s是卷积核每次在空间上移动的步长

p是在输入的四周补充0，确保输出的尺寸

上述的block是串连起来的，上一个block的输出是下一个block的输入，比如block1->block2，block1输出了32张图，然后对这32个图分别做64个卷积核的卷积操作，得到64张输出图，以此类推

最后得到一个256维的向量输入分类头这个板块中，在训练的时候随机丢弃一部分参数dropout过程

训练的时候会根据loss**反向传播**把梯度传到所有的可以学习的参数上面

RNN循环神经网络Recurrent Neural Network

1. 为什么要RNN，CNN有什么缺点

   就是因为CNN无法体现这个序列的顺序性，例如

   it is too red 和 it is red too 在CNN中可能非常相似，无法体现序列的顺序性
2. RNN的原理

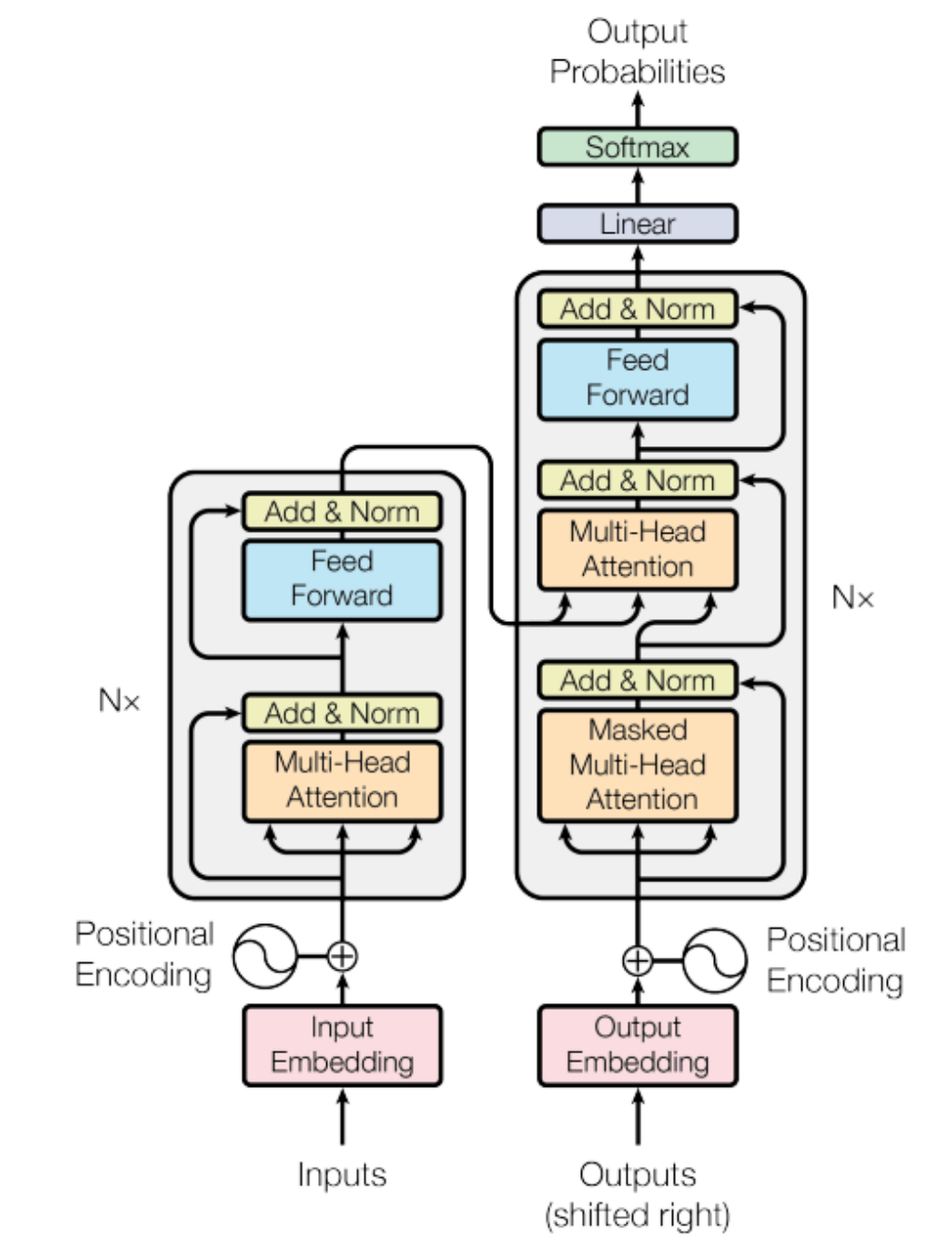
   输入层进入隐藏层，隐藏层输出进入输出层这是经典机器学习的方法

   RNN是把隐藏层的输出继续作为下一次计算的隐藏层的输入
3. RNN的主要的缺点
   1. 记忆短
   2. 计算慢，必须从头到尾，而且不能并行，推理延迟高

Transformer以及注意力机制

model architecture

1. encoder&decoder stack



拿一个进行翻译句子“我爱你”的任务来讲解一下这个transformer架构的执行方式

1. 在训练阶段，准备好的语料作为inputs传入input embedding进行输入嵌入，然后通过positional encoding为他们做位置编码，然后分别计算QKV传入multi-head attention，其中QKV都是向量，d_{model} = 512 d_v= d_k = 64 h = 8
2. 处理结束之后就是一个64维的计算结果，处理之前每个token是各自的token，处理之后每个token的含义对应的向量会向这个文本内容中的对应的含义的方向倾斜，然后匹配对应方向的含义八个头 也

就是一个token对应了8个64维的向量，把这八个拼接在一起成了一个512维的向量之后经过add & norm相加并归一

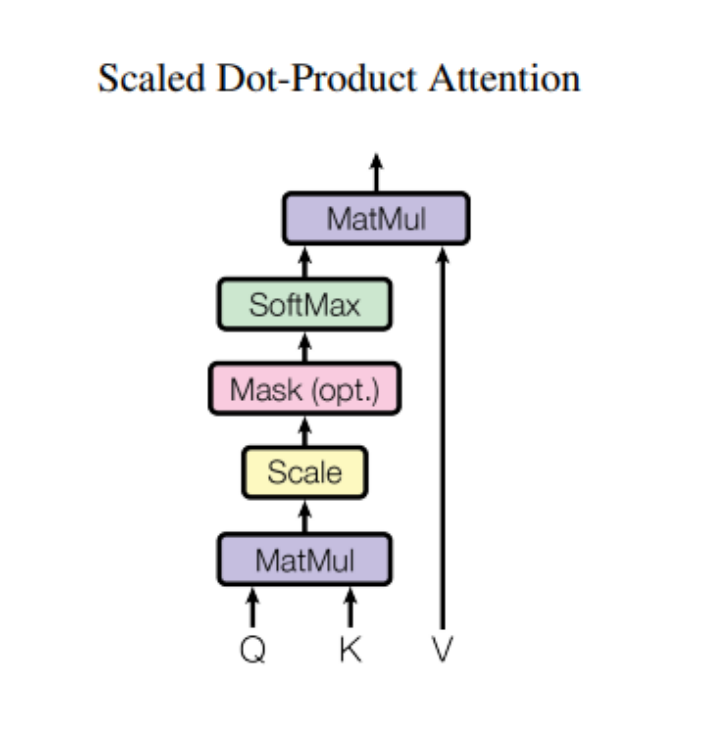
3. 然后进入feed forward全连接前馈网络，进行一个语义的筛选，把512维的向量扩展成2048维，然后经过激活函数ReLU的筛选，筛掉无关维度方向的负值变成0，然后再变成512维度进入下一个环节也就是add&norm 相加是为了保证原始数据还在，避免因为这一子层的处理效果不佳，损失了原来的数据，**归一化是为了稳定训练，加速收敛**

4. 重复这个过程6次，将输出输入到右侧进行输出的部分，初始的状态是outputs为空或者是一个的状态，然后经过一个masked multi-head attention的处理和add & norm的处理，masked和无masked的区别在于multi-head attention可以看到整个句子中的全部的token，而multi-head attention则只能看到已经生成的token，而没有生成的token是无法看到的，也就是可能一句话“我爱你”，在生成你的时候，只能看到“我爱”，这个其实通过上一步的outputs感觉就已经实现了

5. 上一步传入的实际上是Q和K，也就是告诉下一步我要找的内容的向量是什么样的，要的标签是什么样的，计算出来对应的权重之后和现在生成的语句的V做计算，走左边的流程，并重复六次，生成的是下一个可能的token对应的概率，从而得到对应的待生成的token的向量，然后转化成对应的token就生成了真正的一句话，也就是“我爱你”

2. **attention函数**

    1. Scaled Dot-Product Attention



一个注意力函数可以被描述为映射一个query询问和一组kv对到一个output，注意query key value output都是向量形式的

$$Attention(Q,K,V)=softmax(\frac{QK^T}{\sqrt{d_k}})V$$

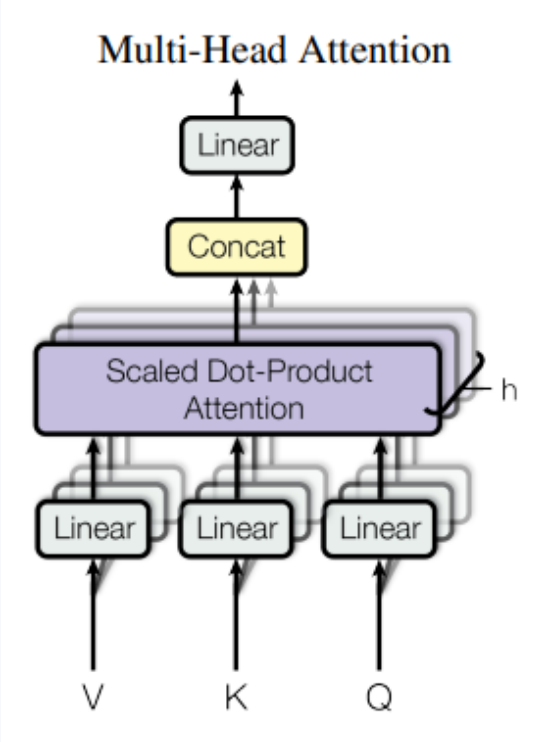
- 比如把一句话用embedding模型给他变成向量模型X
- QKV分别用这个X乘以对应的$W_QW_KW_V$得到最后的QKV，其中这个$W_QW_KW_V$是权重矩阵，通过训练不断调整参数的，一般来讲QKV的形状是一样的，但是允许V的列数不同，用多少维度来表示一个token是可变的
- 实际计算的时候是把多个询问放到矩阵Q里面，KV同样都是矩阵得到的然后一起计算的
- 例如 小明 喜欢 喝水 分成了左边的这三个token
    1. 先把每个token变成一个对应的向量X_i[1,0] [0,1] [1,1]
    2. W_Q W_K W_V 是训练好的对应的权重，这里设成单位矩阵，也就是说QKV在这里都等于X
    3. 把X_i分别和第二个步骤中的矩阵相乘得到QKV都

是3*2的矩阵

4. 现在我们想研究喜欢这个词和哪个词关联更紧密那么query就是[0,1] 然后和对应的key分别 也就是K做点积得到[0,1,1]也就是喜欢和这三个位置的匹配程度

5. 然后对这个[0,1,1]做softmax得到对应的权重，再乘以V，其中权重代表这个词在整个句子中更加关注哪一个词语，attention则表示经过考察这个token对于其他token的关注之后得到的结果

2. Multi-Head Attention



$$MultiHead(Q,K,V) = Concat(head_1, head_2, \ldots, head_h)W^O$$
$$where\ head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$
$$W_i^Q \in R^{d_{model} \times d_k}$$
$$W_i^K \in R^{d_{model} \times d_k}$$
$$W_i^V \in R^{d_{model} \times d_v}$$

1. concat函数会让列变多，用W^O投影回正常大小

2. d_{model}是对于大模型来说你设置一个token转成多少维度的向量,d_k是说对于键值对的键（标签，不同的任务不一样）是多少维度的向量

3. 查询的维度和标签的维度是一样的，结果在这个阶段先表示成d_v维度的，然后返回给模型的其他任务的时候再还原成d_{model}

3. application of attention

具体的应用方法可以见我上面的图解说明，这部分说明就是说一下怎么应用的注意力方法

4. Position-wise Feed-Forward Networks

$$FFN(x) = max(0, xW_1 + b_1)W_2 + b_2$$

1. 第一步 xW1+b1 通过训练得到的W1将512维的向量变成2048维
2. 第二步 激活函数ReLU取大于0的部分,正数代表在这个分量上是符合的，负数代表相反或者无关，取大于0的部分可以筛选掉哪些无关方向的特征
3. 第三步 xW2 + b2 把2048维度转成512维度进入下一个部分

为什么要升维度？变成2048个维度，更加精细的表示这个token的标签，筛选掉无关标签

5. Embeddings and Softmax

1. embeddings : learned embeddings 可学习词嵌入 把token转成向量
对于输入和输出和最后预测用的的线性层的权重矩阵是统一个权重矩阵，
减少参数量、语义表征统一
2. softmax：$softmax(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$ 高分更高低分更低 拉大差距，

并且最终和为1

6. Positional Encoding

这个模型没有循环结构也就是RNN的那套从头到尾按顺序理解的过程，位置编码和词嵌入拥有相同的维度，因此二者可以直接相加，位置编码有多种实现方案，既可以采用可学习型，也可以采用固定常量型。

$$PE_{(pos,2i)} = sin(\frac{pos}{10000^{\frac{2i}{d_{model}}}})$$
$$PE_{(pos,2i+1)} = cos(\frac{pos}{10000^{\frac{2i}{d_{model}}}})$$

why self-attention

从三个维度上来讲解transformer RNN CNN之间的性能

1. total computational complexity per layer 计算时间复杂度的问题
2. the amount of computation that can be parallelized 并行问题，RNN无法并行
3. path length between long-range dependencies in the network 网络里面的长距离依赖